FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	**Alvin Christian S. Panio** **Faith Nemeño**
Section:	**AN43-BSCSSE**
Professor:	**Crisola Tan**
Date:	**October 25, 2026**

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) is an intelligent problem-solving approach used in **Knowledge Representation and Reasoning (KRR)**. Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, including retrieval, reuse, revision, and retention.
2. **Represent cases effectively** using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.
3. **Implement a similarity assessment method** to identify relevant past cases based on defined attributes.
4. **Adapt retrieved solutions** to meet the needs of a new problem scenario based on the chosen case study.
5. **Demonstrate learning in CBR** by saving the newly solved case into the system's case base.
6. **Develop collaboration skills** through pair programming and joint problem analysis.
7. **Apply knowledge representation and reasoning concepts** to real-world problems in disaster response, legal reasoning, or intelligent tutoring.

Materials Needed

1. Hardware
    - A laptop or desktop computer
    - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice
- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

4. Reference Materials

Lecture notes on:

- Knowledge Representation & Reasoning
- Case-Based Reasoning cycle
- Similarity measures

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)
    - Google Drive for shared documentation
    - GitHub/GitLab for code collaboration

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Please ensure that you are grouped as pairs in the People tab.
3. Select only one case study from the list below.
4. Include clear comments in your code to describe its logic.
5. Each program should display the correct input, process, and output.
6. Ensure that your script displays clear and readable outputs.
7. Compile all Python codes and corresponding outputs into one PDF file.
8. Save it as **TA3_lastname1_lastname2.pdf**

    (Example: TA2_Patilano_Loverez.pdf)
9. Submit the **python file** and **PDF** file.

📌 Step-by-Step Instructions

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*
3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** stored as dictionaries, JSON objects, arrays, or any structure.

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- Attribute matching
- Weighted scoring
- Simple rule-based comparison

Output should show: (sample)

✓ Similarity score or ranking

✓ The most relevant past case retrieved

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.

This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** Adjust evacuation plans based on population size
- **Legal DSS:** Modify arguments due to new digital evidence
- **ITS:** Give personalized feedback based on student errors

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ "New case saved" or similar confirmation

✓ Updated case list

7. Display clear program outputs

Your output must include:

- New problem description
- Retrieved similar case
- Adapted final solution
- Updated case base

8. Submission Requirements

Submit the following:

A. Program File

- Python

B. Short Documentation (1–2 pages)

Include:

- Title of chosen case study
- Description of your case base
- Explanation of similarity method
- Adaptation logic
- Screenshots of program execution

C. Pair Reflection (5–7 sentences)

Discuss:

- What each partner contributed
- What challenges you encountered
- What you learned about CBR

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- Past cases include typhoons, floods, and earthquakes.
- New event: *Typhoon with expected flooding in urban areas.*
- System retrieves similar events: *Typhoon Ursula (2019)* and *Typhoon Odette (2021)*.
- It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.
- After execution, the system stores new outcomes for improved future reasoning.

Learning Outcome:

CBR helps governments respond faster by using *experiential knowledge* rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- online theft

- digital fraud
- unauthorized bank transfers

Adaptation:
The reasoning engine adjusts argument patterns to match the new technical context.
Learning:
After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System
A CBR system identifies the learning struggles of students. Student submits a math solution with repeated errors. System retrieves past students with similar error patterns (e.g., misusing formulas). It adapts feedback and personalized lessons based on successful interventions. **Learning:** System improves by retaining new student profiles and outcomes.

Your Code - Laboratory Tasks ⇩

Completed work

Selected case study: Intelligent Tutoring System. Pair members: Alvin Christian S. Panio and Faith Nemeño. The following pages contain the two-page documentation, pair reflection, complete Python program, and tests.

Intelligent Tutoring System

Short documentation 1 of 2

This program recommends math practice from comparable past error patterns. It demonstrates retrieval, reuse, revision and retention with five illustrative cases. It does not infer mastery from real student work; topic, error type, mastery and repeated errors are entered as structured features.

Case representation

Each Python dictionary has an ID, problem, solution and outcome. The problem stores topic, error, mastery (integer 0–100) and repeats (integer 1–5). The solution stores feedback and exercise count; adapted cases also include support and review. New cases record the retrieved source ID. JSON provides persistence between runs.

C01 Linear equations; sign error; mastery 30; repeats 3; 4 exercises.

C02 Linear equations; distribution error; mastery 45; repeats 2; 3 exercises.

C03 Linear equations; balance error; mastery 60; repeats 2; 3 exercises.

C04 Fractions; denominator error; mastery 35; repeats 4; 4 exercises.

C05 Area; formula error; mastery 50; repeats 2; 3 exercises.

Similarity and retrieval

Only cases with the same topic AND error are eligible. This prevents unrelated feedback from being reused. Within that set, the highest weighted similarity wins; equal scores are resolved by case ID. If there is no eligible case, the program requests teacher guidance and saves no recommendation.

$$\text{Score} = 0.25T + 0.45E + 0.20(1 - |M_{new} - M_{old}|/100) + 0.10(1 - |R_{new} - R_{old}|/4).$$

T and E are 1 for matching topic and error, otherwise 0. M is mastery and R is repeated errors. The weights total 1; all terms lie between 0 and 1. Because eligibility requires matching topic and error, those two terms are fixed at 0.70 within the ranked set. The numeric features distinguish otherwise comparable cases.

Adaptation and revision

Practice count equals the retrieved count plus the difference in repeated errors, limited to 2–8 exercises. Mastery below 40 adds a fully worked example with fading hints; higher mastery starts with a hint and independent work. A reviewer then supplies a correction or confirmation. The demo adds substitution into the original equation as a check.

Retention and limits

Interactive mode requires confirmation of a successful intervention and an outcome description before saving. The demo uses a clearly illustrative outcome. Identical problem/solution pairs are not duplicated. JSON is written atomically and reloaded; the updated list and a fresh retrieval show the retained case being used. Weights and thresholds are educational design choices, not validated teaching policy.

Intelligent Tutoring System execution

Short documentation 2 of 2

Run: python tutoring_cbr.py --demo --case-base demo_cases.json

Use a new JSON filename for the initial five-case demonstration. The actual captured run below retrieves C01 at 0.965, adapts four exercises to five, and retains C06. The saved case base is reloaded with six cases; the same query then retrieves C06 at 1.000.

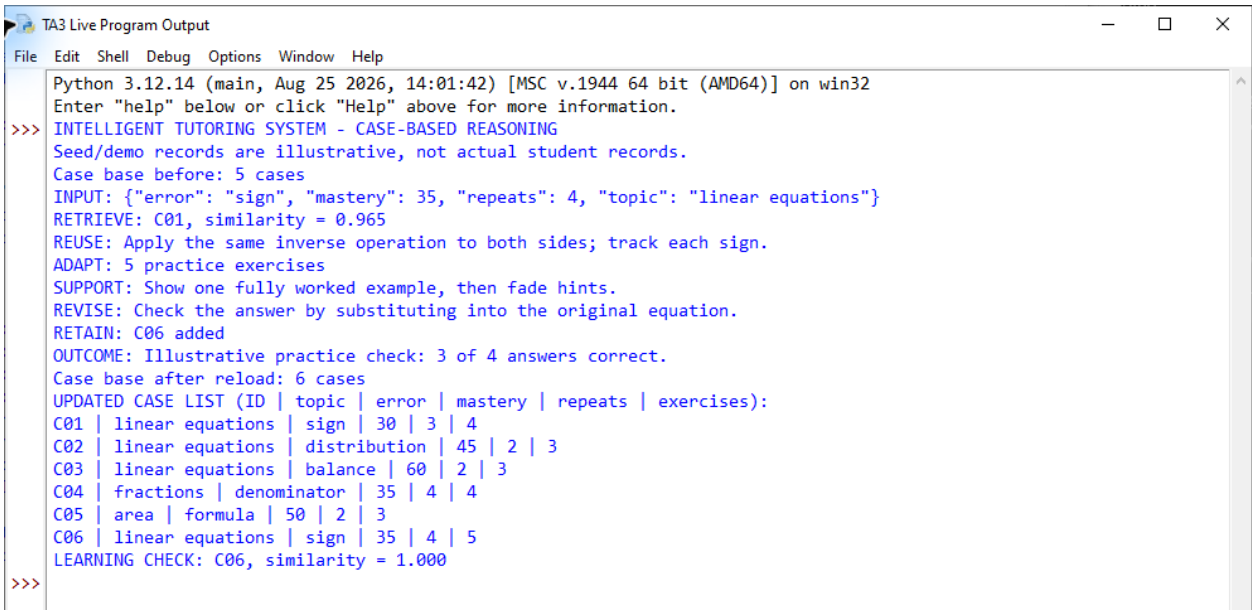


Figure 1. Actual screenshot of tutoring_cbr.py running in Python IDLE. It shows retrieval, adaptation, revision, retention, the updated case list, and the learning check. Seed records and the practice outcome are illustrative.

Worked similarity check

For the new sign-error case (mastery 35, repeats 4) and C01 (30, 3), the score is 0.25 + 0.45 + 0.20 × 0.95 + 0.10 × 0.75 = 0.965. After retention, C06 matches all four features and scores 1.000. This demonstrates case reuse and storage, not a measured improvement in student performance.

Execution and validation

The program uses Python 3.9 or later and the standard library. Interactive mode is python tutoring_cbr.py --case-base my_cases.json. Nine automated tests passed, covering retrieval, invalid and unsupported input, adaptation, bounded practice, review/outcome requirements, durable retention, duplicate prevention and deterministic ties.

Pair reflection

As partners, Alvin Christian S. Panio and Faith Nemeño share equal responsibility for understanding our program, checking its results, and reviewing the documentation. We represent each case with a problem, a solution and an outcome so that the stored experience has enough context to be useful. A challenge in our design is preventing a similar mastery score from retrieving feedback for an unrelated error, so we require the topic and error type to match before comparing scores. We also use a duplicate check to keep repeated runs from storing the same problem and solution again. Our demonstration retrieves C01, adapts the practice from four exercises to five, records a review, and retains C06 for later retrieval. The example shows us that CBR depends on relevant cases and reviewed outcomes, and that saving a new case demonstrates learning in the case base without proving improved student performance.

Complete Python program

File: tutoring_cbr.py. The following listing is the complete delivered program.

```
"""TA3: a small case-based math tutor. All seed and demo data are illustrative."""
import argparse
import copy
import json
from pathlib import Path


def make_case(cid, topic, error, mastery, repeats, feedback, exercises):
    return {"id": cid, "problem": {"topic": topic, "error": error,
            "mastery": mastery, "repeats": repeats},
            "solution": {"feedback": feedback, "exercises": exercises},
            "outcome": "Successful intervention in illustrative seed data."}


SEEDS = [
    make_case("C01", "linear equations", "sign", 30, 3,
              "Apply the same inverse operation to both sides; track each sign.", 4),
    make_case("C02", "linear equations", "distribution", 45, 2,
              "Multiply every term inside brackets, including its sign.", 3),
    make_case("C03", "linear equations", "balance", 60, 2,
              "Write each operation on both sides before simplifying.", 3),
    make_case("C04", "fractions", "denominator", 35, 4,
              "Find a common denominator before adding numerators.", 4),
    make_case("C05", "area", "formula", 50, 2,
              "Identify the shape and label dimensions before choosing a formula.", 3),
]


def validate(problem):
    """Reject malformed feature values before distance calculations."""
    for key in ("topic", "error"):
        if not isinstance(problem.get(key), str) or not problem[key].strip():
            raise ValueError(f"{key} must be nonempty text")
    for key, low, high in (("mastery", 0, 100), ("repeats", 1, 5)):
        value = problem.get(key)
        if type(value) is not int or not low <= value <= high:
            raise ValueError(f"{key} must be an integer from {low} to {high}")
```

```
def similarity(new, old):
    """Weighted score in [0, 1]; numeric distances use fixed domain ranges."""
    validate(new)
    validate(old)
    return (0.25 * (new["topic"] == old["topic"])
            + 0.45 * (new["error"] == old["error"])
            + 0.20 * (1 - abs(new["mastery"] - old["mastery"]) / 100)
            + 0.10 * (1 - abs(new["repeats"] - old["repeats"]) / 4))


def retrieve(problem, cases):
    """Only reuse cases with the same topic AND error; break ties by ID."""
    validate(problem)
    candidates = [(similarity(problem, c["problem"]), c) for c in cases
                  if all(problem[k] == c["problem"][k]
                         for k in ("topic", "error"))]
    if not candidates:
        raise ValueError("No matching topic/error case; ask a teacher for a new case.")
    return sorted(candidates, key=lambda item: (-item[0], item[1]["id"]))[0]


def adapt(problem, case):
    """Reuse past feedback, then adjust practice and scaffolding."""
    solution = copy.deepcopy(case["solution"])
    delta = problem["repeats"] - case["problem"]["repeats"]
    solution["exercises"] = max(2, min(8, solution["exercises"] + delta))
    solution["support"] = ("Show one fully worked example, then fade hints."
                           if problem["mastery"] < 40 else
                           "Start with a hint, then solve independently.")
    return solution


def revise(solution, review_note):
    """A reviewer supplies an explicit correction or confirmation."""
    if not review_note.strip():
        raise ValueError("A review note is required before retention")
    revised = copy.deepcopy(solution)
    revised["review"] = review_note.strip()
    return revised


def retain(cases, problem, solution, outcome, source_id):
    """Retain only reviewed solutions with a recorded successful outcome."""
    validate(problem)
    if not solution.get("review") or not outcome.strip():
        raise ValueError("Reviewed solution and successful outcome are required")
    for case in cases:
        if case["problem"] == problem and case["solution"] == solution:
            return case, False  # Re-running the same demonstration is idempotent.
    next_id = max((int(c["id"][1:]) for c in cases), default=0) + 1
    case = {"id": f"C{next_id:02d}", "problem": copy.deepcopy(problem),
            "solution": copy.deepcopy(solution), "outcome": outcome.strip(),
            "retrieved_from": source_id}
    cases.append(case)
    return case, True


def load_cases(path):
```

```
    if not path.exists():
        return copy.deepcopy(SEEDS)
    cases = json.loads(path.read_text(encoding="utf-8"))
    if not isinstance(cases, list):
        raise ValueError("Case base must contain a JSON list")
    ids = set()
    for case in cases:
        validate(case["problem"])
        cid = case["id"]
        if not (isinstance(cid, str) and cid.startswith("C")
                and cid[1:].isdigit()) or cid in ids:
            raise ValueError("Case IDs must be unique C-prefixed integers")
        ids.add(cid)
        solution = case["solution"]
        if (not isinstance(solution.get("feedback"), str)
                or not solution["feedback"].strip()
                or type(solution.get("exercises")) is not int
                or not 2 <= solution["exercises"] <= 8):
            raise ValueError("Invalid stored solution")
    return cases


def save_cases(path, cases):
    """Replace atomically after a complete JSON write."""
    path.parent.mkdir(parents=True, exist_ok=True)
    temporary = path.with_suffix(path.suffix + ".tmp")
    temporary.write_text(json.dumps(cases, indent=2) + "\n", encoding="utf-8")
    temporary.replace(path)


def run(path, demo=False):
    cases = load_cases(path)
    print("INTELLIGENT TUTORING SYSTEM - CASE-BASED REASONING")
    print("Seed/demo records are illustrative, not actual student records.")
    print(f"Case base before: {len(cases)} cases")
    if demo:
        problem = {"topic": "linear equations", "error": "sign",
                   "mastery": 35, "repeats": 4}
    else:
        print("Available topic/error pairs: " + "; ".join(sorted({
            f'{c["problem"]["topic"]}/{c["problem"]["error"]}' for c in cases})))
        problem = {"topic": input("Topic: ").strip().lower(),
                   "error": input("Error: ").strip().lower(),
                   "mastery": int(input("Mastery (0-100): ")),
                   "repeats": int(input("Repeated errors (1-5): "))}
    print("INPUT:", json.dumps(problem, sort_keys=True))
    score, matched = retrieve(problem, cases)
    print(f'RETRIEVE: {matched["id"]}, similarity = {score:.3f}')
    print("REUSE:", matched["solution"]["feedback"])
    solution = adapt(problem, matched)
    print(f'ADAPT: {solution["exercises"]} practice exercises')
    print("SUPPORT:", solution["support"])
    review = ("Check the answer by substituting into the original equation."
              if demo else input("Reviewer correction/confirmation: "))
    solution = revise(solution, review)
    print("REVISE:", solution["review"])
    if demo:
        success = True
```

```
        outcome = "Illustrative practice check: 3 of 4 answers correct."
    else:
        success = input("Was the intervention successful? (yes/no): ").lower() == "yes"
        outcome = input("Describe observed outcome: ") if success else ""
    if not success:
        print("RETAIN: deferred; revise the intervention with a teacher.")
        return
    learned, added = retain(cases, problem, solution, outcome, matched["id"])
    save_cases(path, cases)
    print(f'RETAIN: {learned["id"]} ' + ("added" if added else "already stored"))
    print("OUTCOME:", learned["outcome"])
    print(f"Case base after reload: {len(load_cases(path))} cases")
    print("UPDATED CASE LIST (ID | topic | error | mastery | repeats | exercises):")
    for case in load_cases(path):
        p, s = case["problem"], case["solution"]
        print(f'{case["id"]} | {p["topic"]} | {p["error"]} | '
              f'{p["mastery"]} | {p["repeats"]} | {s["exercises"]}')
    new_score, new_match = retrieve(problem, load_cases(path))
    print(f'LEARNING CHECK: {new_match["id"]}, similarity = {new_score:.3f}')


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--case-base", type=Path, default=Path("cases.json"))
    parser.add_argument("--demo", action="store_true")
    args = parser.parse_args()
    try:
        run(args.case_base, args.demo)
    except (ValueError, KeyError, TypeError, OSError) as error:
        parser.exit(1, f"Input/case-base error: {error}\n")
```

Automated test source

File: test_tutoring_cbr.py. Run: python -m unittest -v test_tutoring_cbr

```
"""Behavior tests for retrieval, adaptation, revision and durable learning."""
import copy
import tempfile
import unittest
from pathlib import Path
from tutoring_cbr import (SEEDS, adapt, load_cases, retain, retrieve, revise,
                          save_cases, similarity, validate)


class CBRTests(unittest.TestCase):
    def setUp(self):
        self.cases = copy.deepcopy(SEEDS)
        self.problem = {"topic": "linear equations", "error": "sign",
                        "mastery": 35, "repeats": 4}

    def test_weighted_score_and_retrieval(self):
        score, case = retrieve(self.problem, self.cases)
        self.assertEqual(case["id"], "C01")
        self.assertAlmostEqual(score, .965)
        self.assertAlmostEqual(similarity(self.problem, self.problem), 1)

    def test_unseen_topic_or_error_has_no_false_match(self):
        for change in ({"topic": "geometry"}, {"error": "unknown"}):
            with self.assertRaises(ValueError):
                retrieve(self.problem | change, self.cases)
        with self.assertRaises(ValueError):
            retrieve(self.problem, [])

    def test_adaptation_is_personalized_without_mutating_source(self):
        before = copy.deepcopy(self.cases[0])
        solution = adapt(self.problem, before)
        self.assertEqual(solution["exercises"], 5)
        self.assertIn("fully worked", solution["support"])
        self.assertEqual(before, self.cases[0])
        solution = adapt(self.problem | {"mastery": 80}, before)
        self.assertIn("independently", solution["support"])

    def test_practice_is_bounded(self):
        case = copy.deepcopy(self.cases[0])
        case["solution"]["exercises"] = 8
        self.assertEqual(adapt(self.problem, case)["exercises"], 8)
        case["solution"]["exercises"] = 2
        self.assertEqual(adapt(self.problem | {"repeats": 1}, case)["exercises"], 2)

    def test_review_and_outcome_are_required(self):
        solution = adapt(self.problem, self.cases[0])
        with self.assertRaises(ValueError):
            revise(solution, " ")
        with self.assertRaises(ValueError):
            retain(self.cases, self.problem, solution, "success", "C01")
        with self.assertRaises(ValueError):
            retain(self.cases, self.problem, revise(solution, "checked"), "", "C01")

    def test_retention_persists_and_improves_retrieval(self):
```

```
        solution = revise(adapt(self.problem, self.cases[0]), "checked")
        new, added = retain(self.cases, self.problem, solution, "success", "C01")
        self.assertTrue(added)
        self.assertEqual(new["id"], "C06")
        with tempfile.TemporaryDirectory() as directory:
            path = Path(directory) / "cases.json"
            save_cases(path, self.cases)
            reloaded = load_cases(path)
            self.assertEqual(len(reloaded), 6)
            score, matched = retrieve(self.problem, reloaded)
            self.assertEqual(matched["id"], "C06")
            self.assertAlmostEqual(score, 1)

    def test_duplicate_is_not_retained_twice(self):
        solution = revise(adapt(self.problem, self.cases[0]), "checked")
        retain(self.cases, self.problem, solution, "success", "C01")
        _, added = retain(self.cases, self.problem, solution, "success", "C01")
        self.assertFalse(added)
        self.assertEqual(len(self.cases), 6)

    def test_invalid_features_and_corrupt_storage_are_rejected(self):
        for change in ({"mastery": -1}, {"mastery": 101}, {"repeats": 0},
                       {"repeats": 6}, {"mastery": True}, {"topic": ""}):
            with self.assertRaises(ValueError):
                validate(self.problem | change)
        with tempfile.TemporaryDirectory() as directory:
            path = Path(directory) / "cases.json"
            path.write_text("not json", encoding="utf-8")
            with self.assertRaises(ValueError):
                load_cases(path)

    def test_ties_are_deterministic(self):
        twin = copy.deepcopy(self.cases[0])
        twin["id"] = "C99"
        self.assertEqual(retrieve(self.problem, [twin, self.cases[0]])[1]["id"], "C01")


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Test execution result

All nine tests passed. Captured unittest summary:

```
----------------------------------------------------------------------
Ran 9 tests in 0.061s

OK
```